

TERRAFORM · A PRACTICAL INTRODUCTION

Terraform: Infrastructure as Code, End to End

Write it. Review it. Build it. Repeat it.

WHY INFRASTRUCTURE AS CODE

Clicking through consoles doesn't scale

Manual setup

- Settings drift between environments.
- Changes lack a reviewable diff.
- Repeating a build depends on memory.

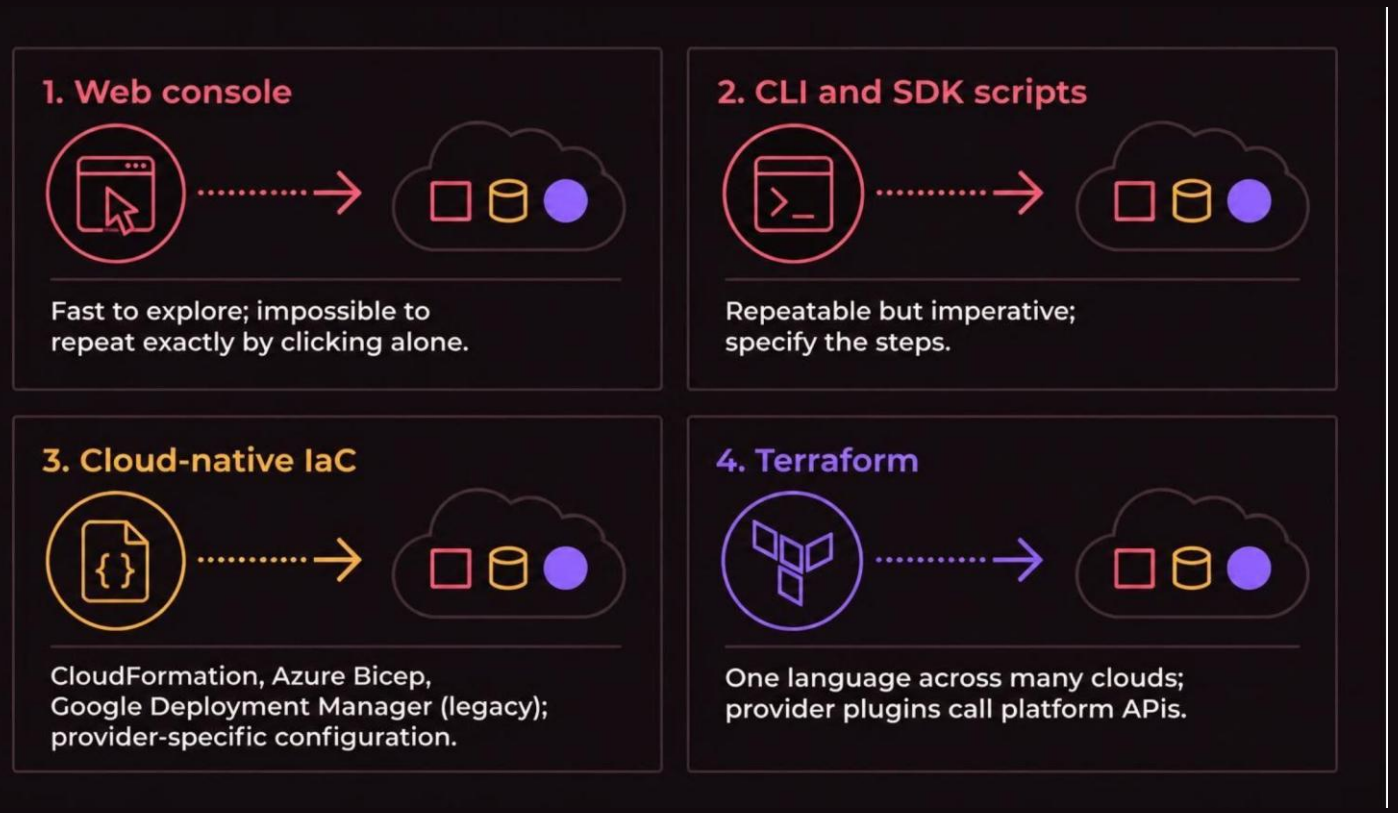
Infrastructure as code

- Version every change.
- Test and review before applying.
- Share a repeatable configuration.



THE TOOLING LANDSCAPE

Four ways to provision cloud resources



Google Deployment Manager is a legacy example; use current platform guidance for new deployments.

THE MENTAL MODEL

IaC concepts that shape every tool

Declarative

Describe the end state, not the steps to reach it.

Idempotent

Run again with unchanged inputs and reality; no new change is needed.

Version-controlled

Infrastructure changes go through pull requests, with a reviewable history.

Drift

Real infrastructure diverges from code, often after an out-of-band change.

GETTING STARTED

Install Terraform and read your first file

- Install the single binary via package manager or download; run `terraform -version`.
- Write HCL configuration in `.tf` files.

Core blocks: `terraform`, `provider`, `resource`, `variable`, `output`.

MAIN.TF · A FIRST RESOURCE

```
terraform { required_version = ">= 1.6" }
resource "terraform_data" "hello" { input =
"Hello, infrastructure" }
output "message" { value =
terraform_data.hello.output }
```

PLATFORM INTEGRATION

Providers connect Terraform to platforms

PROVIDER REQUIREMENTS + CONFIGURATION

```
terraform { required_providers { azurerm
= { source = "hashicorp/azurerm",
version = "~> 4.0" } } } provider
"azurerm" { features {} }
```

- Plugins connect to AWS, Azure, GCP, Kubernetes and thousands more from the Terraform Registry.
- Constrain versions in `required_providers` for predictable runs.
- Supply credentials through the environment or workload identity, never hard-coded.

Commit `.terraform.lock.hcl` to preserve selected provider versions.

CONFIGURATION ANATOMY

Resources are the things you build

```
resource "azurerm_resource_group" "main" {
  name     = "rg-web-dev"
  location = "westeurope"
}

resource "azurerm_virtual_network" "main" {
  name                = "vnet-web"
  resource_group_name =
azurerm_resource_group.main.name
  location            =
azurerm_resource_group.main.location
  address_space       = ["10.0.0.0/16"]
}
```

The reference creates an implicit dependency: resource group before virtual network.

Type → name → arguments

- `azurerm_virtual_network` identifies the resource type.
- `main` is its local name.
- Arguments describe the desired settings.
- Data sources read existing objects without managing them.

A CONFIGURABLE INTERFACE

Variables in, outputs out

01

Inputs

Type, default, description, validation.

02

Configuration

Use `var.name` to configure resources.

03

Outputs

Expose IP addresses and URLs.

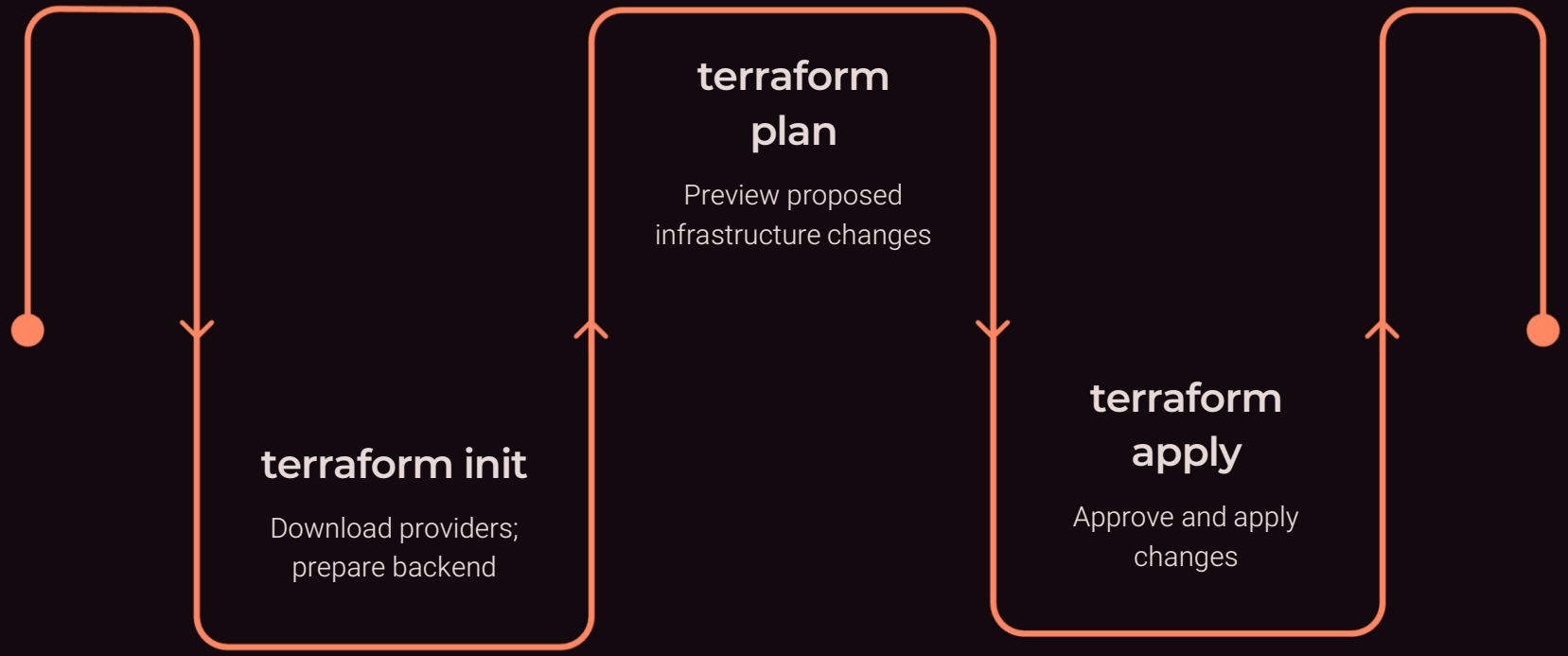
```
variable "instance_type" {
  type        = string
  default     = "Standard_B2s"
  description = "Web VM size"
}
```

```
output "web_url" {
  value =
"http://${azurerm_public_ip.web.ip_a
ddress}"
}
```

Set inputs with `.tfvars`, `-var` or `TF_VAR_` environment variables.

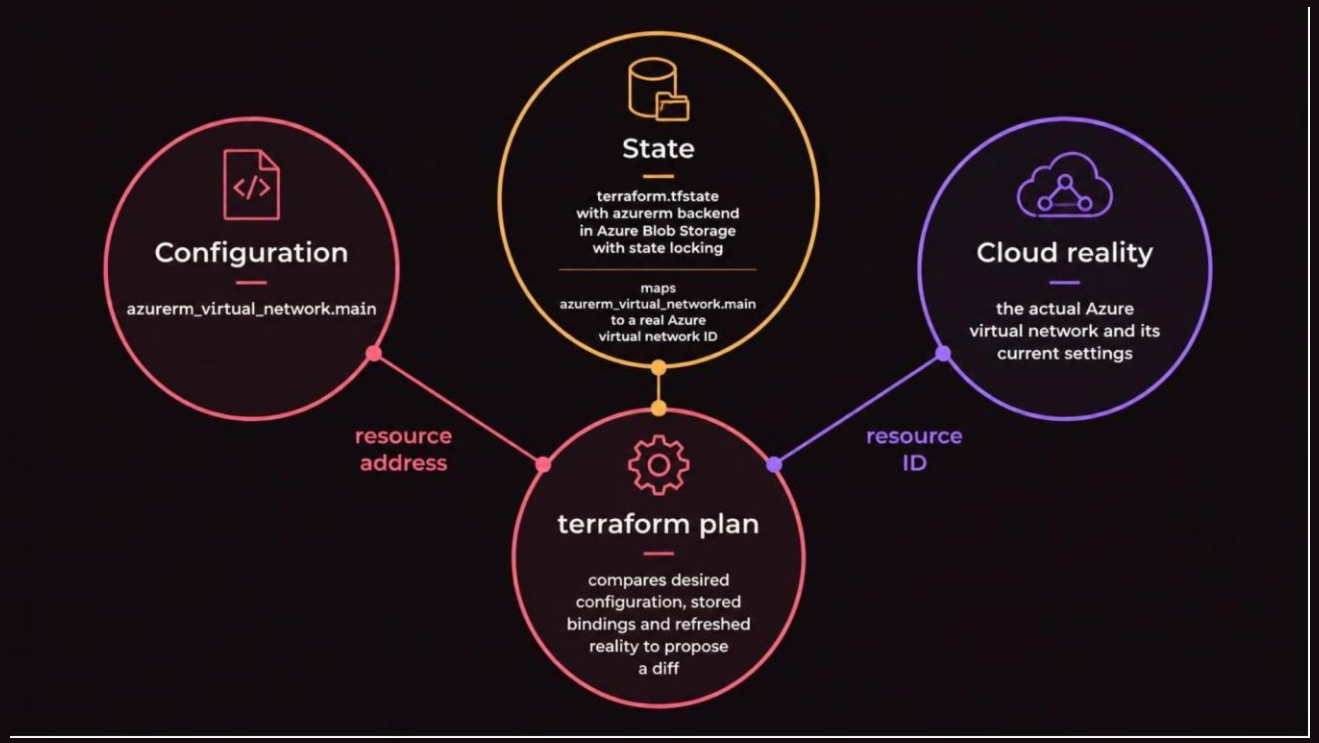
THE EXECUTION CYCLE

Init, plan, apply: the core workflow



TRACKING MANAGED OBJECTS

State is Terraform's memory



State can hold secrets. **Never commit it to Git.**

Inspect with `terraform state list` and `terraform state show`; adopt existing objects with `terraform import`.

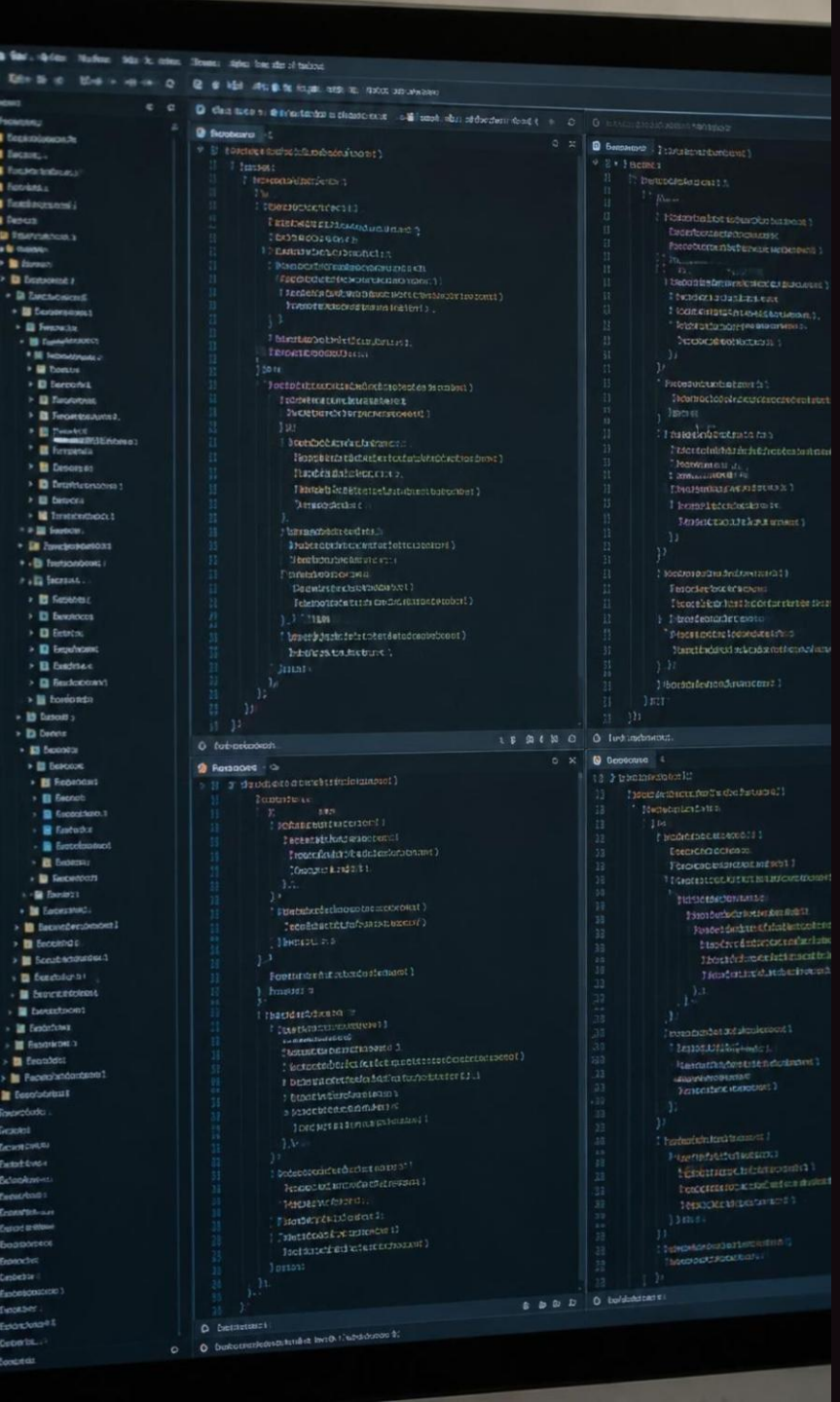
TEAM-SAFE OPERATIONS

Share state safely with remote backends

- Store state in Azure Blob Storage, GCS or Terraform Cloud.
- Enable supported locking to prevent concurrent writers.
- Separate independent state for dev, staging and prod.

Restrict access, encrypt storage and retain state versions for recovery.

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-tfstate"
    storage_account_name = "sttfstateteam"
    container_name       = "tfstate"
    key                  =
"dev/terraform.tfstate"
  }
}
```



REUSABLE INFRASTRUCTURE

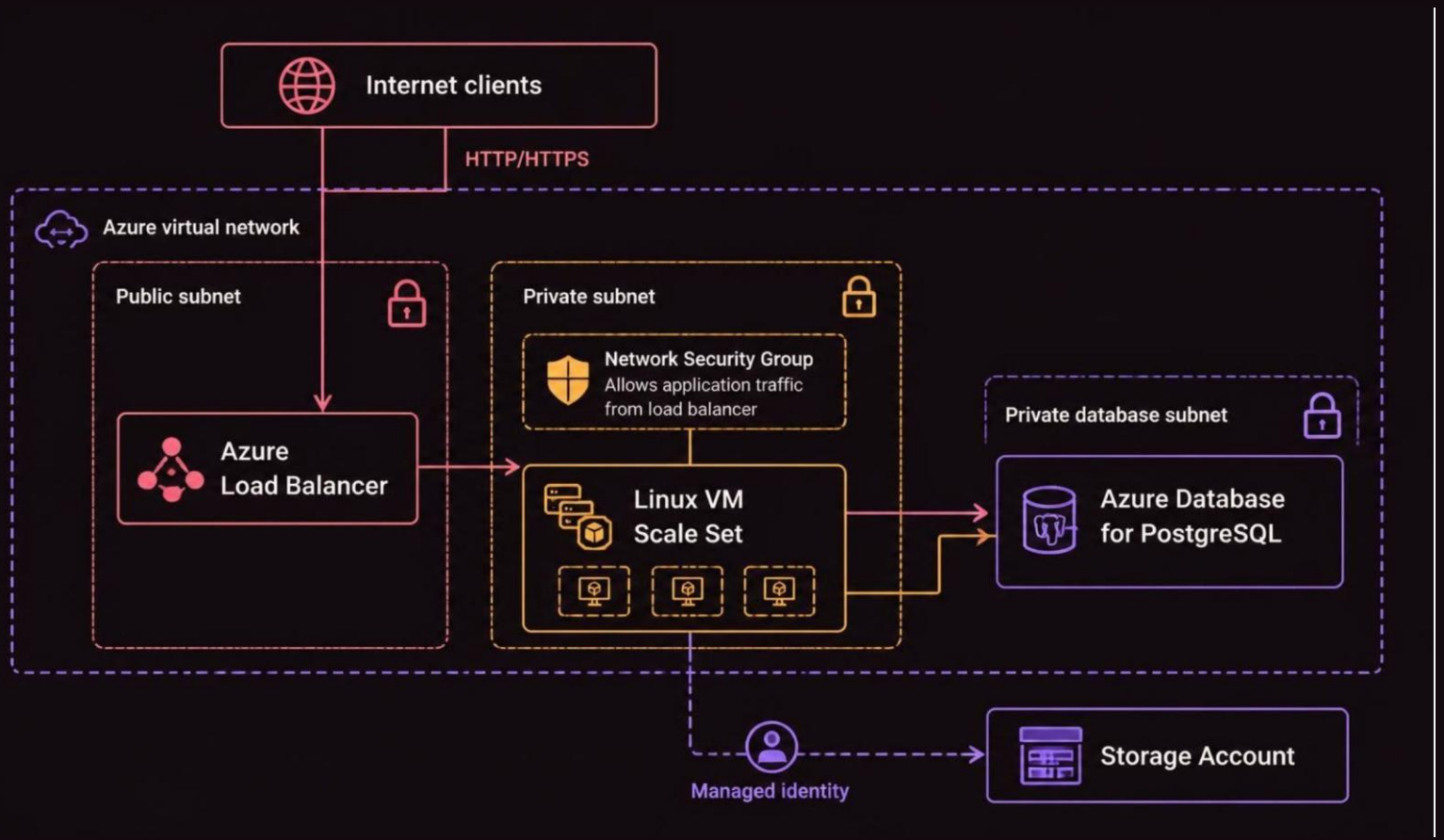
Modules turn patterns into reusable building blocks

- A module is a folder of `.tf` files with inputs and outputs.
- Call it with a `module` block and a `source`.
- Reuse public registry modules or your own team library.

```
module "network" {
  source = "./modules/network"
  cidr   = "10.0.0.0/16"
}
# Reference an output: module.network.vnet_id
```

PUTTING THE PIECES TOGETHER

The end-to-end build: a web tier on the cloud



THE BUILD WALKTHROUGH

From empty folder to running infrastructure

01

Write the configuration

Define provider, variables, network module and compute resources.

02

Initialize and review

`terraform init`, then `terraform plan`. Read the proposed diff.

03

Apply and inspect

`terraform apply`, then `terraform output` for the load balancer URL.

04

Change one input

Re-plan after changing a variable; inspect only the resulting delta.

05

Clean up

`terraform destroy`. Confirm deletion and check for residual costs.

YOUR NEXT IMPLEMENTATION

Start small, then standardize

01 **Begin with one resource**

Use a remote backend from the start.

02 **Extract repeatable patterns**

Grow into modules and per-environment workspaces.

03 **Make review routine**

Add plan reviews to your CI pipeline before applying.

Keep state protected. Keep changes reviewable.

